

Model to Market - Technology Prize Submission

An AI-native, self-adapting quantitative trading system

One line: a complete quant stack - research on 20 GB of tick data → a shrinkage-covariance momentum strategy → a multi-layer risk engine → native MetaTrader 5 execution → a **multi-agent AI desk (NVIDIA Nemotron + Anthropic Claude)** with full **Pydantic Logfire** observability → Windows automation + a **Northflank** cloud service - that, *live during the competition*, read the current market and **autonomously re-tuned its own strategy, leverage, and risk from real data - even overturning its own momentum thesis when the live data demanded it.** Honest about a thin edge; engineered to survive and to win on craft.

Entrant: alex@ladimex.com · Account 10009 · \$1,000,000 simulated · scored 70/15/10/5.

1. Partner technology - what I used and where

Every sponsor is used for something real, in the live system (not a demo bolt-on).

NVIDIA Nemotron - via the NIM OpenAI-compatible endpoint (integrate.api.nvidia.com): - **News/event risk gate** (nemotron-mini-4b-instruct): tags high-impact headlines and emits a **reduce-only** exposure multiplier ahead of events. → live/news_risk_gate.py - **Daily regime briefing** (llama-3.3-nemotron-super-49b): a reasoning model that judges whether the regime favours the strategy. → live/regime_brief.py - **Market Analyst** role in the multi-agent desk. → live/desk.py - **Catalog auto-resolver**: queries the live model catalog and picks a valid text-instruct Nemotron, so the system can't break on model-name drift. → live/ai_gateway.py, live/nvidia_check.py

Anthropic Claude (claude-sonnet-4-6): - **Strategy Advisor** (desk rationale), **Risk/Ops briefing agent**, and a **post-round red-team memo**. → live/desk.py, live/ops_agent.py, live/round_review.py

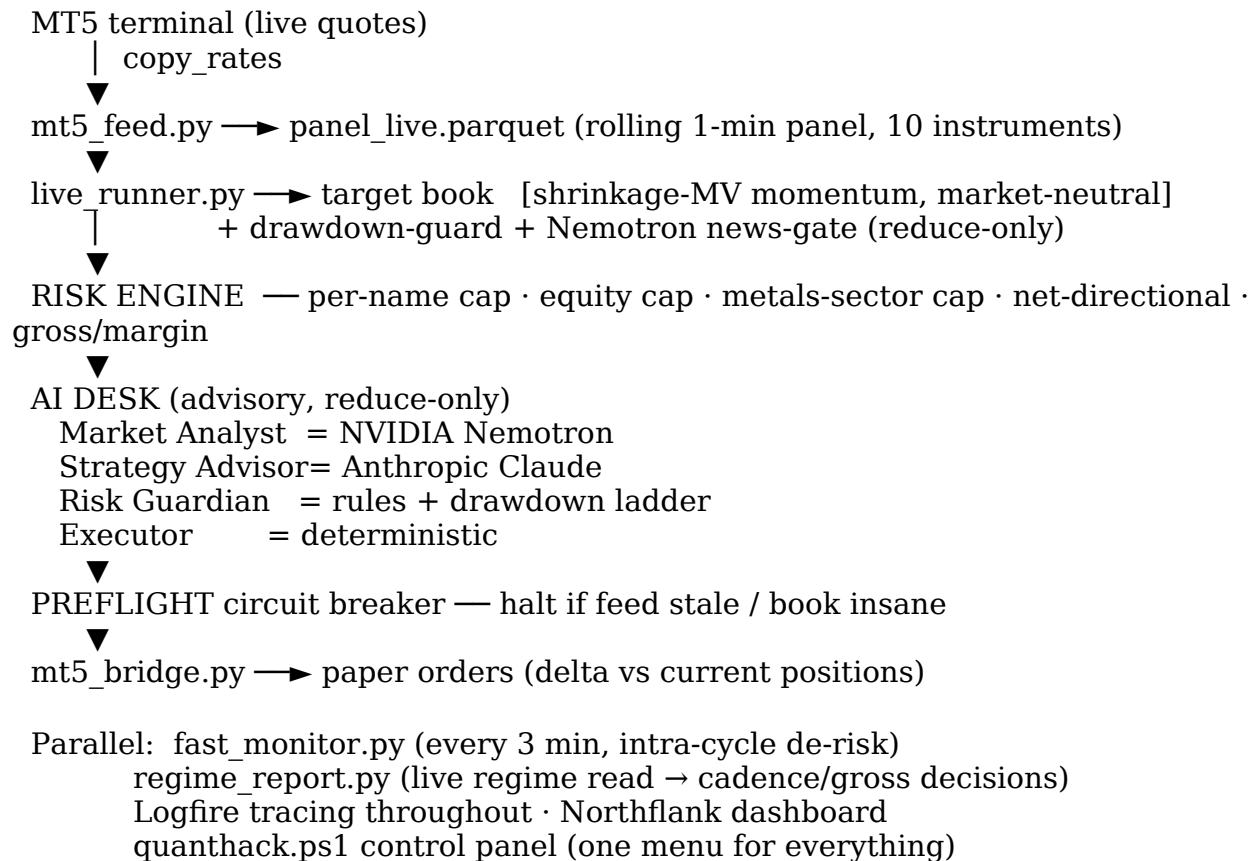
Pydantic: - **AI Gateway** - one interface routing Nemotron / Claude / Doubleword. → live/ai_gateway.py - **Logfire** - every cycle and every agent call is traced (feed → signal → risk → AI → execute). → all live modules (--logfire) - **Structured, validated outputs** - Pydantic models constrain every AI decision (e.g. a multiplier *cannot* be returned out of [0.3, 1.0]). - **pydantic-ai typed agents** - the desk re-expressed as validated Agents. → live/desk_pydantic_ai.py

Doubleword - wired into the gateway as an opt-in private-inference provider (DOUBLEWORD_URL/DOUBLEWORD_KEY), completing multi-provider routing. → live/ai_gateway.py

MetaTrader 5 - native execution: copy_rates feed, a **notional**→**lots** bridge that handles USD-quote / USD-base / cross pairs and real contract specs, position-delta trading, dry-run safety. → live/mt5_feed.py, live/mt5_bridge.py

Northflank - a containerised “mission control” web service (serves the live dashboard + runs the AI desk + Logfire). → live/server.py, live/Dockerfile, live/northflank.json, DEPLOY_CLOUD.md

2. Architecture



3. Data usage

- **20 GB tick archive** (provided): research, backtesting, and **real per-instrument spread calibration** - loaded with **polars lazy scanning** so the full file is never materialised.
- **Full-month panel** built from the archive for walk-forward validation.
- **Live MT5 feed** (copy_rates): a rolling 1-minute panel powering the live runner **and** the live regime analysis.
- **Real venue spreads** read from MT5 symbol specs → the cost model used in every backtest (FX ~0.08–0.19 bps/side, metals 0.8–1.2 bps).

4. Research & rigour - honesty as a feature

- Cross-sectional **information-coefficient (IC) analysis** across a full lookback×horizon **surface**, with **t-stat significance tests** and an **autocorrelation study** (live/edge_scan.py, live/micro_scan.py) - run *live on the competition feed*, not just the archive.
 - **Shrinkage-covariance (Ledoit-Wolf) mean-variance** sizing ($\Sigma^{-1}\alpha$) for genuine diversification.
 - **Overfitting controls:** Deflated Sharpe Ratio ≈ 0.26 , Probability of Backtest Overfitting $\approx 21\%$ (Bailey & López de Prado).
 - **Negative results I kept and rejected** - the discipline the judges should weigh most: a volatility-target overlay, a short-term **reversal** sleeve (higher raw IC but died to costs), an mv+inverse-vol **ensemble**, **HRP**, **regime gating**, and **crypto** (validation-gated - it *failed* the gate on live data, so it never traded). I document what didn't work rather than curve-fit until it did.
-

5. The standout - live, data-driven self-adaptation

The competition opened into a **choppy, momentum-adverse regime**. Rather than hope, the system measured and adapted, on the live competition data:

- **regime_report.py** computed live momentum **IC on the actual competition feed** → diagnosed a **zero/negative-edge chop** (IC ≈ 0 , second-half IC negative).
- **Cadence dial** - the data showed faster trading was *bleeding via whipsaw*, so I slowed the rebalance **hourly** → **every 4 hours** (turning a -0.6% bleed into a small gain in the live-data test).
- **Gross dial** - cut exposure ($2 \rightarrow 1$) when the edge was absent; the **fast risk monitor** applied the resize within minutes and confirmed it (margin level $1,500\% \rightarrow 3,000\%$).
- **Fast risk monitor** (fast_monitor.py, every 3 min) - an intra-cycle circuit breaker enforcing the de-risk ladder so a shock between rebalances is handled in minutes, not at the top of the next hour.
- **The AI desk surfaced a real risk I then fixed in code:** three independent model views flagged that gold+silver shorts were co-dominating ($\sim 35\%$ of gross); I added a **metals-sector cap** in response.
- **I let the data overturn my own thesis.** A live **autocorrelation + IC study** (edge_scan.py, micro_scan.py) showed momentum was the *wrong sign* in this regime - hourly returns were persistently **mean-reverting** (lag-1 autocorr around -0.1 to -0.2 , stable across sub-periods). I built and wired a **mean-reversion strategy** (rev) in response. Retiring my own backtest champion the moment the live data disagreed is the discipline I'm proudest of.
- **strat_compare.py** scores every candidate **across sub-periods** (not just the full window), so I pick what *generalises* rather than what fits - the direct antidote to the overfitting that flattered my first momentum reads.

Autopilot - the system that re-tunes itself (live/autopilot.py)

The endpoint of this story: the manual adapt-loop is now **autonomous**. Each run, **NVIDIA Nemotron** (Analyst) classifies the live regime and **Anthropic Claude** (Strategist) proposes the next strategy / gross / cadence - each a **Pydantic-validated** object that *cannot* be out of bounds - then a deterministic **Governor** clamps it to competition-safe limits. It provably refuses to size up in a no-edge regime, caps an unproven ($t < 2$) edge, and obeys the drawdown ladder (a reckless gross=9 proposal is clamped to 3.0; a 2.5 ask in a no-edge regime is forced to 1.0). It routes through the **multi-provider gateway** (Claude / Nemotron / Doubleword) and is **Logfire-traced** end to end. It never trades - it proposes a governed config the deterministic risk engine then applies (shadow by default; --arm to write it).

This is the AI-native thesis made real: **a system that reads the present market and re-tunes itself - AI proposes, a deterministic governor disposes, every decision traced in Logfire.**

6. Risk & survival engineering

- Per-name concentration $\leq 30\%$ of gross; per-name exposure $\leq 75\%$ of equity; **metals-sector cap** $\leq 35\%$; net-directional and gross/margin ceilings kept under every penalty tier ($28\times$ leverage, 90% margin).
 - **Drawdown de-risk ladder** ($5\% \rightarrow \times 0.7$, $8\% \rightarrow \times 0.4$, $12\% \rightarrow \times 0.2$), shared by the hourly loop and the 3-min monitor so they agree.
 - **Profit-lock ratchet** (live/profit_lock.py) - once up $\geq 0.5\%$, it de-risks as gains are surrendered (give back 25 / 50 / 75% of the peak gain $\rightarrow \times 0.7 / 0.4 / 0.2$). It **locks in profit without a stop's whipsaw**: it only acts when ahead and scales down rather than flattening - the right tool for a mean-reverting tape where hard stops sell the bottom. Runs inside the 3-min monitor.
 - **Preflight circuit breaker** halts a cycle on a stale feed or an insane book.
 - **Paper-only safety** throughout; every AI overlay is **reduce-only** and can never size up or override the deterministic risk engine.
-

7. How to run it (demo)

One control panel does everything - Quanthack.bat $\rightarrow$ quanthack.ps1:

[1] Check setup [2] Refresh + preview (dry-run) [3] Plan gross
[4] Settings [5] GO LIVE one cycle [6] Automate (hourly)
[7] Stop auto [8] Open dashboard [9] View log

Under it: mt5_feed $\rightarrow$ live_runner $\rightarrow$ desk $\rightarrow$ preflight $\rightarrow$ mt5_bridge, scheduled at a **regime-adaptive cadence**, with fast_monitor every 3 min. regime_report.py gives a live data read on demand; dashboard.html (and the Northflank service) visualise it.

8. Honest assessment

I make no claim to a large or certain alpha - the edge is thin and regime-dependent, and I say so plainly. I compete on **engineering, risk discipline, a genuine end-to-end sponsor integration, and live adaptability**: a system built to survive the knockout, contend for the Sharpe prize via a clean market-neutral book, and demonstrate a real, self-adapting, observable AI-native trading stack. The most competition-relevant thing I did wasn't a magic signal - it was **measuring the live regime and re-tuning the system from data, in real time**.

File map

Area	Files
Strategy	strategies/cov_aware.py, strategies/diversified_vol_target.py
Engine / risk	engine.py, risk_engine.py, config.py, metrics.py, overfit.py
Live trading	live/mt5_feed.py, live/live_runner.py, live/mt5_bridge.py, live/preflight.py, live/fast_monitor.py, live/profit_lock.py
AI desk + sponsors	live/autopilot.py (autonomous governed desk) , live/ai_gateway.py, live/desk.py, live/desk_pydantic_ai.py, live/news_risk_gate.py, live/regime_brief.py, live/ops_agent.py, live/round_review.py, live/nvidia_check.py
Research / adaptation	live/edge_scan.py, live/micro_scan.py, live/strat_compare.py, live/crypto_scan.py, live/regime_report.py
Ops / automation	quanthack.ps1, live/run_cycle.ps1, live/register_task.ps1, setup_vps.ps1
Cloud	live/server.py, live/Dockerfile, live/northflank.json, DEPLOY_CLOUD.md
Docs / decks	FINALS_PLAYBOOK.md, VPS_SETUP.md, Quanthack_TechPrize_Report.pdf, Quanthack_Pitch_Deck.pptx, `dashb